

Embedded Real-Time Systems

Recap

Henri Heyden

stu240825

Kiel University

Summer Term 2026

Disclaimer

I do not guarantee this summary to contain every part of the lecture, as it is only supposed to summarize the things I personally found the most important when going through the slides.

This document was compiled on October 3, 2026.

Introduction

The Embedded Real-Time Systems lecture of Kiel University by Reinhard von Hanxleden is based on *Introduction to Embedded Systems—A Cyber-Physical Systems Approach* by Edward Lee and Sanjit Seshia¹. Embedded systems are computers whose main purpose is the interaction with a physical process. This interaction is commonly implemented using sensors, actuators, and physical dynamics. A real-time system is broadly defined by the notion, that not only the correctness of the output of a machine matters, but also the timing in which they arrive. The term cyber-physical system denotes computerized systems which communicate over a network and interact with a physical process.

In this book and lecture it is postulated, that embedded systems should be created using an iterative process, which is called *Modeling, Design, Analysis*.

Modeling The process of gaining a deeper understanding of a system through environmental imitation. Models specify *what* a system does.

Design The structured creation of artifacts. Designs specify *how* a system does what it does.

Analysis The process of gaining a deeper understanding of a system through dissection. Analysis specifies *why* a system does what it does, or fails to do so.

This approach has the important benefit, that we can implement mechanisms to compile embedded systems software based on a mathematical model of the system and environment. The compiler can perform a model analysis during translation.

¹https://ptolemy.berkeley.edu/books/leeseseshia/releases/LeeSeshia_DigitalV2_3.pdf

Contents

1	Modeling Dynamic Behaviours	7
1.1	Continuous Dynamics	7
1.2	Discrete Dynamics	9
1.3	Hybrid Automata	11
1.4	State Machine Compositions	11
1.5	Dataflow Models	14
1.5.1	Synchronous Dataflow	14
1.5.2	Other Variants	15
1.6	Timed Models of Computation	15
1.6.1	Event-Triggered Models	15
1.6.2	Time-Triggered Models	16
1.6.3	Multiform Notion of Time	16
1.6.4	Eager Semantics	16
2	Design of Embedded Systems	17
2.1	Sensors and Actuators	17
2.2	Memory Architectures	18
2.3	Input and Output	19
2.3.1	Polling	20
2.3.2	Interrupts	20
2.4	Concurrency	20
2.4.1	Scheduling	21
2.4.1.1	Rate Monotonic Scheduling	21
2.4.1.2	Earliest Due Date	22
2.4.1.3	Earliest Deadline First	22
2.4.1.4	Latest Deadline First	22
2.4.2	Scheduling Anomalies	22
2.4.2.1	Richards Anomalies	23

3	Analysis and Verification	25
3.1	Correctness and Model Checking	25
3.1.1	Linear Temporal Logic	26
3.2	Equivalence and Refinement	26
3.3	Reachability Analysis and Model Checking	28
3.3.1	Graph Traversal Algorithms	28
3.3.2	Using Temporal Logic	28
3.3.3	Liveness Properties and Finding Acceptance Cycles	29
3.4	Quantitative Analysis	29
3.4.1	Measuring Runtime	29
3.5	Achieving High Code Quality	29
3.5.1	Testing	30
3.5.1.1	Behaviour-Driven Development	30

Chapter 1

Modeling Dynamic Behaviours

In this chapter we will look at different dynamic mechanisms to mathematically model environment and system using so-called *actor-based* systems.

1.1 Continuous Dynamics

Continuous dynamics directly mimic continuous physical processes. An example where continuous dynamics are used to control an embedded system is a proportional integral differential (PID) controller in a helicopter which is used to cancel out the torque generated by the main rotor using the side rotor.

Generally, the dynamical models in this chapter act on functions between time and some value.

Definition 1.1.1 (General Physical Axioms). In the model we define position and angle as functions on time.

$$\begin{aligned}x &: \mathbb{R} \rightarrow \mathbb{R}^3 \\ \theta &: \mathbb{R} \rightarrow \mathbb{R}^3\end{aligned}$$

From this we can obtain the (angular) velocity and acceleration as the respective first and second differential.

$$\ddot{x} = \partial \dot{x} = \partial \partial x$$

If we add a constant mass $M \in \mathbb{R}$, we can furthermore obtain force by Newton's second law.

$$F(t) = M \cdot \ddot{x}(t)$$

If we have a radius $r \in \mathbb{R}$ and an inertia tensor $I : \mathbb{R} \rightarrow \mathbb{R}^{3 \times 3}$, torque can be found for a point-mass using either

$$T(t) = r \cdot F(t)$$

or

$$T(t) = \partial \left(I(t) \cdot \dot{\theta} \right).$$

For one dimension and static inertia tensor, this boils down to

$$T_y(t) = I_{yy} \cdot \ddot{\theta}_y(t)$$

which can be written in integral form

$$\dot{\theta}(t) = \dot{\theta}(0) + 1/I_{yy} \int_0^t T_y(\tau) d\tau.$$

These physical processes are computationally hard to model, as a computer can not possibly predict the future, or solve complex integrals to direct equations. This is why in the *actor model of systems*, the model calculates general feedback integrals by using approximations.

Definition 1.1.2 (Actor Model of Systems). In continuous dynamics, the actor model of systems is defined by using so-called *actors*, which are functions on functions of time, which we call *signals*. For example

$$S : (\mathbb{R} \rightarrow \mathbb{R}) \rightarrow (\mathbb{R} \rightarrow \mathbb{R})$$

is an actor.

As already mentioned, a computer can not predict the future, but this model helps by keeping the history in mind. The *integrator* actor $I : (\mathbb{R} \rightarrow \mathbb{R}) \rightarrow (\mathbb{R} \rightarrow \mathbb{R})$ can be thus defined.

$$I(x)(t) = x(0) + \int_0^t \dot{x}(\tau) d\tau$$

Here the actor has a parameter $x(0)$, the initial value, and produces a value function $x : \mathbb{R} \rightarrow \mathbb{R}$ based on initial value and acceleration $\dot{x}$, which is the input of the actor.

An actor can also have multiple signals as inputs, see $\oplus : (\mathbb{R} \rightarrow \mathbb{R})^2 \rightarrow (\mathbb{R} \rightarrow \mathbb{R})$ with

$$\oplus(x, y)(t) = x(t) + y(t).$$

1.2 Discrete Dynamics

In discrete dynamics we do not look at signals which provide input at every moment, but which are only *present* sometimes.

Definition 1.2.1 (Pure and discrete signals). A signal is called *pure* if it is of form $\mathbb{R} \rightarrow \{\text{absent}, \text{present}\}$ or $\mathbb{R} \rightarrow \{\text{absent}\} \cup S$ for some set S . We define a pure signal x to be *discrete* iff

$$\exists f \in \mathbb{Z}^{\{t \in \mathbb{R} | x(t) \neq \text{absent}\}}, f \text{ is injective} : \forall t_1, t_2 \in T : t_1 < t_2 \Rightarrow f(t_1) < f(t_2)$$

In other words, a signal x is discrete iff there exists an one-to-one function on the set of all moments, where x was not absent to $\mathbb{Z}$, which is order-preserving. All elements of this set have to be ordable on $\mathbb{Z}$. Alternatively, if we assume, that there exists a minimal or first time in which x was present, the function can also map to $\mathbb{N}$. Note, that countability does not imply discreteness!

We call an actor on discrete signals discrete, iff the output signal is also discrete. Note, that this is not trivial, the composition of two discrete signals does not have to be discrete.

Reactions of a discrete system are triggered by the environment in which the actor operates—it *reacts*. In the case that an actor produces an output, which happens on reaction, we call this output *valuation of type V* , where V is the domain of the domain of the actor.

As mentioned, we consider a discrete actor to react if, but not only if it changes its internal state, this can be done in different ways. Finite state machines (FSM) can be used as a *refinement* of a discrete actor to define its behaviour.

Definition 1.2.2 (Finite State Machines). FSMs are mathematical constructions consisting of *states*, *transitions*, and an *initial* state. A FSM starts in the initial state. Transitions are of form $s, g \rightarrow s', o$, where s is the state the transition starts in, g is the *guard* of the transition, s' is the state the transition ends in, and o is the (optimal) output valuation the FSM produces, when the transition is taken.

A FSM which is the refinement of a discrete actor changes its state if a guard of a possible transition is true. The actor produces a value, if the transition it takes has an output valuation, else it is absent.

The guard of a transition can be based on input signals. E.g., $p_1 \wedge \neg p_2 \vee p_3 = 3$ is true, when p_1 is present, and p_2 is absent, or when p_3 is present with value 3.

A transition can be set *default*, which means it is enabled if no non-default transition is enabled. This is highlighted by using a dotted line. Every FSM defined implicit default self transitions. I.e., if no transition is enabled, the FSM neither reacts nor changes its state—any present input is disregarded, but the system is still well defined.

It is also possible to define non-deterministic transitions, which is indicated through a red line. These transitions enable multiple transitions to be taken at the same time. If a FSM contains a non-deterministic transition, it is called a non-deterministic FSM. In this case, the set of resulting states becomes the permutation set of all possible states.

We extend the FSM model using the following model.

Definition 1.2.3 (Extended State Machines and Hierarchical State Machines). Extended state machines (ESMs) add more features in comparison to FSMs. The extension brings variables $v \in \mathcal{V}$ and enables transitions to use *set actions* $v := \sigma$, where σ is the new value for v . σ can be any right-hand side of an output action—it can thus also be based on input signals.

ESMs can also be *hierarchical* which defines refinements in states as other state machines. A state with such a refinement is in a substate defined by the underlying state machine. The semantics are defined, such that first the refinement (if any) reacts, and then the top-level state machine reacts. These two reactions are counted as one reaction as their transitions are simultaneous even if one is evaluated after the other.

Transitions are redefined on ESMs with hierarchical refinements, where the normal transition is called *history transition*, as it remembers the previous state of a refinement if its container state is reached. There can also be *reset transitions*, denoted with a hollow arrow, which reset to the starting state of a state refinement. And finally, there are also *preemptive transitions*, denoted with a red circle at the start of it, which enforce that a refinement does not react, if a state with an outgoing preemptive transition contains it.

Flattening hierarchical state machines with history transitions defines a composed state space according to the semantics of the transitions. If a refinement reacts, it should reach another composed state with the current top-level state, if it does not change.

Flattening hierarchical state machines with reset transitions do not need a composed state space, as they do not remember the refinement state, when transitioning on top-level.

Flattening hierarchical state machines with preemptive transitions is done by removing transitions and guards which make a substate react.

We also define some basic terms on SMs.

Definition 1.2.4. A transition is called *stuttering*, iff it is enabled and produces an absent output. A *behaviour* is a sequence of non-stuttering steps. A *trace* is the record of inputs, states, and outputs in a behaviour. A *computation tree* is a graphical representation of all possible traces.

SMs provide a way to mathematically represent the environment of the system and the system itself. They can be used for designing requirements and analysing the safety of the system.

Currently we do not have defined, *when* a reaction occurs, i.e., when to read the next input values for the signals. This is vitally important, as we can define ESMs with a default self transition, which increases the value of a variable—the value becomes dependent on the *rate* of reactions, which is currently undefined. We will go over this notion of time in Section 1.6.

1.3 Hybrid Automata

In Sections 1.1 and 1.2 we defined continuous and discrete automata, in this section we will define *hybrid automata*, which combine both approaches by refining individual states with continuous actors.

In hybrid automata, signals do not have to be discrete—they are allowed to be continuous-time signals. This again begs the question *when* a reaction happens. In this model, we define the actor to react *asap*—which depends on the simulation speed, when evaluating the model.

For example, we can define a continuous actor using a hybrid model which contains one state s with variable $v = 0$ and default transition $s, \text{true} \rightarrow s, v, \dot{v} := 1$. This hybrid automata defines a continuous actor with no input signal, which outputs the linear function.

Definition 1.3.1 (Hybrid Automata). We formally define hybrid automata to be ESMs with continuous input and output signals. Moreover, it is allowed to depend the value of a variable in *continuous time*, by setting derivatives of variables in a set action. Reactions occur as soon as possible.

Hybrid automata also define refinements to states which are themselves continuous actors.

1.4 State Machine Compositions

It is possible to combine different SM-refined (discrete) actors together on a top level. There are four different compositions with different semantics.

Synchronous side-by-side composition Multiple SMs in parallel with reaction only possible when all SMs react.

Asynchronous side-by-side composition Multiple SMs in parallel with non-deterministic reaction when any SM can react.

Cascade composition Multiple SMs chained together with output signals as input signals for another SM.

Feedback composition Recursive connection of a SM which sets input signals to own output signals.

For the following definitions, we will look at compositions between two SM actors A and B with states S_A, S_B , input valuations (or guards) i_A, i_B , output valuations o_A, o_B , initial states I_A, I_B , and update function $u_A : S_A \times i_A \rightarrow S_A \times o_A$, for B analogous, to represent the transitions.

Definition 1.4.1 (Synchronous Side-by-side Composition). In synchronous side-by-side compositions for SMs A and B , we define the set of states as $S := S_A \times S_B$, the inputs $i := i_A \times i_B$, outputs $o := o_A \times o_B$, initial state $I := (I_A, I_B)$, and update function $u : S \times i \rightarrow S \times o$,

$$u((s_A, s_B), (v_A, v_B)) = ((s'_A, s'_B), (v'_A, v'_B)),$$

with $(s'_A, v'_A) = u_A(s_A, v_A)$ and $(s'_B, v'_B) = u_B(s_B, v_B)$.

In other words, we just concatenate all input, states, outputs, and derive next output/state by calculating the next state and output for both SMs, and combining them.

This means that, even though A and B may have one input pair, for which A reacts and B produces an absent value, the combined SM reacts. This is because B reacts without producing a value—the reactions are triggered by the environment.

Definition 1.4.2 (Asynchronous Side-by-side Composition). In the asynchronous side-by-side composition, the environment chooses if either A or B react¹. This is called *interleaving semantics*. Thus we have the same definition as in the synchronous version, but with the rule that the update function $u : S \times i \rightarrow S \times o$ is defined as

$$u((s_A, s_B), (v_A, v_B)) = \begin{cases} ((s'_A, s_B), (v'_A, \text{absent})) & \text{when } A \text{ reacts,} \\ ((s_A, s'_B), (\text{absent}, v'_B)) & \text{when } B \text{ reacts.} \end{cases}$$

This is realized using non-deterministic compositions when calculating the composite actor.

Definition 1.4.3 (Cascade Composition). The cascade composition of A and B is considered to be the simultaneous and instantaneous reaction of both actors, where the output signal of A is the input signal of B . This is of course only possible if the expression *type checks*, i.e., when $o_A \subseteq i_B$.

To construct the composition machine, first form the state space as the cross product of the state spaces of A and B , and then determine which transitions are taken under what conditions. It is important to remember that the transitions are simultaneous, even when one logically causes the other.

The last possible composition between two actors is the feedback composition. To define the semantics of such a composition, the question again begs which actor should react *first*. Tackling this requires defining another model of computation, the *synchronous-reactive* model of computation.

The semantics of a feedback composition should be similar to the cascade composition in the way, that they are simultaneous and instantaneous. The difference being, that the composed actor does not oscillate in propagating data from one end to the other, but

¹It is also possible, that both actors react. These semantics are not discussed in this model.

computes a solution instantaneous, where output matches input. Thus, in a feedback composition of two actors, they react at the same time.

By reorganizing any composition of actors, it can be shown that there is an equal simple feedback composition.

Definition 1.4.4 (Feedback Composition). Realizing a feedback composition between A and B needs $o_A \subseteq i_B \wedge o_B \subseteq i_A$. As mentioned, any composition can also be rewritten as a feedback composition using one actor, thus we only need to consider $o_C \subseteq i_C$ if C is the refined feedback composition of A and B , which is equivalent to the statement prior².

The goal is finding the *fixed point* signal $s : \mathbb{N} \rightarrow o_C \cup \{\mathbf{absent}\}$ which flows from the output of C to its input. s must hold Equation (1.1) to be a fixed point, where $C : (\mathbb{N} \rightarrow i_C \cup \{\mathbf{absent}\}) \rightarrow (\mathbb{N} \rightarrow o_C \cup \{\mathbf{absent}\})$ is the actor function of C .

$$C(s) = s \quad (1.1)$$

To find s , we consider the *firing function* $f_C : \mathbb{N} \rightarrow (i_C \cup \{\mathbf{absent}\} \rightarrow o_C \cup \{\mathbf{absent}\})$ of C which is a recomposed version of the actor function to determine the signal at reaction $n \in \mathbb{N}$. This transforms Equation (1.1) into Equation (1.2), which is also called a fixed point.

$$\forall n \in \mathbb{N} : s(n) = (f_C(n))(s(n)) \quad (1.2)$$

To find fixed points in arbitrary feedback compositions, we can deduce the simple feedback composition, as mentioned. Many simplified feedback compositions can shown to be *well-formed* which implies the existence of an unique fixed-point. In this case, we can construct a resulting actor based on the fixed point s .

Definition 1.4.5 (Well-formedness). When finding a fixed point with Equation (1.2), there may be two problems. The fixed point may not exist, or it is not unique. In either case, the system is called *ill-formed*—the feedback composition is unstable. If this is not the case, the system is called *well-formed*.

As well-formedness is not equivalent to the existence of a fixed point, there are ill-formed compositions. This is the case when the system has multiple fixed points. Nevertheless, it is possible to find unique fixed points for many actors using an algorithm to test for *constructiveness*.

Algorithm 1 (Determining Constructiveness).

²It has to be noted, that for this case, there are two different versions of C , when composing it, such that there is a connection between A and B inside the composed actor. The inner signal can also be mapped externally and vice versa. For this example we consider a version where A and B are not connected in the body of C but using a signal which is the two signals concatenated.

1. Start with $s(n)$ unknown.
2. Determine $(f_C(n))(s(n))$ by looking at transitions, starting from the initial state and looking if there exists a value which is output by all transitions. This is needed, as the input at time n is unknown.
3. Repeat 2. until all values in $s(n)$ become known or no more progress can be made.
4. If unknown values remain, the model is not constructiveness, else it is.

It is vitally important to remember **not** to search for a fixed point by matching all inputs and outputs for all transitions. While this is a correct approach, for arbitrary state machine actors with big o_C this can take too much time. This is an exhaustive search—which may be impossible if o_C has infinite types. To demonstrate this, look at Figure 6.6 in LeeShia³. When applying Algorithm 1 to this example, it does not find a fixed point in state $s1$, as $1 \neq \text{absent}$. There is the fixed point **absent** though, which is derived from the transition from $s1$ to $s2$, which can not be found using the algorithm—this model is well-formed but not constructive!

1.5 Dataflow Models

As opposed to previous semantics, in *dataflow models* a reaction is not simultaneous—if data flows from an actor A to an actor B , it is put into a FIFO buffer in between while A produces, and B consumes. There are different dataflow semantics, in this lecture we will mainly look at *synchronous dataflow* (SD).

1.5.1 Synchronous Dataflow

The goal in SD is finding schedules for firing actors, such that all token productions and consumptions are balanced, else the consumption buffer may overflow, or run out of tokens. To help with this task, we can define *balance equations* between connected actors.

Definition 1.5.1 (Balance Equations). Suppose we have $n \in \mathbb{N}$ actors $[n]$ with $m \in \mathbb{N}$ connectors. Define the connection matrix $\Gamma \in \mathbb{N}^{m \times n}$ where Γ_{i*} is the vector which defines connector i . The value Γ_{ij} is positive if connector i is connected with j and j produces tokens; it is negative if j consumes tokens; it is zero, if j is not connected through connector i .

The *balance equation* of the SD system is finding $q \in \mathbb{N}^n$, such that $\Gamma \times q = \vec{0}$. A non-zero solution for this equation tells how often in relation each actor must fire.

If the rank of Γ is $n - 1$, there exists a non-trivial solution for the balance equation. If it is n , which is the only other possibility, there does not exist a non-trivial solution.

³https://ptolemy.berkeley.edu/books/leeseshia/releases/LeeSeshia_DigitalV2_3.pdf#figure.6.6

As a reminder, the rank of Γ can be determined, by applying Gaussian elimination and reducing as many rows as possible. The count of non-zero values in the diagonal of the resulting matrix is the rank of Γ .

If a solution is found for a balance equation, there exists a schedule for the system by decomposing the vector into a sum of unity vectors. E.g., solution $(1, 2, 1)^T$ is decomposed as $(1, 0, 0)^T + (0, 1, 0)^T + (0, 1, 0)^T + (0, 0, 1)^T$, which becomes the schedule $(1; 2; 2; 3)$. For this it is advised to reduce to the minimal solution by scaling it to minimal length with integer values.

1.5.2 Other Variants

In *dynamic dataflow* production/consumption rate depend on data, which makes the problem undecidable in means of finding a schedule or proving that a schedule runs with bounded memory.

There also exists *structured dataflow* or *homogeneous* SDF, which is a restriction of dynamic dataflow to make scheduling decidable.

Another variant are *Petri nets* which use *places* to store tokens one-by-one which move using transitions to other places. Each place can only store one token, which can be used to model different programs. Petri nets with finite possible states are equivalent to finite SMs.

1.6 Timed Models of Computation

In this section we will go over different mechanisms on when to trigger reactions/computations in discrete dynamics, synchronous reactive, or hybrid models. To simplify for now, we will assume discrete time, which postulates that outputs are computed instantaneous when its inputs are ready—there is no computation time, or it is negligible.

The goal of this section is to define behaviour and implementation of a *tick function*, which determines, when the model should react.

1.6.1 Event-Triggered Models

In event-triggered models, reactions only happen on outside events, e.g., a sensor-read button press. In our model this would be an input signal which is present. The tick function is defined as an event trigger, which sends a tick, exactly when any signal becomes present.

The problem with this mechanism is, that reactions suddenly can not be triggered by internal events. As an example, consider an ESM with initial state s . This state also has a refinement based on variable $x := 0$ and $\dot{x} := 1$. Now we define a transition to state s with guard $x = 1$, output x , and set action $x = 0$. If there is no input signal, or the input signal is not present directly after $t = 1$, the ESM will not output x . Even worse, if the ESM reacts at

$t = 2$, it will output 2, as the value as progressed further, than it should have, as the ESM can not react by itself.

1.6.2 Time-Triggered Models

In time-triggered architectures, actors react according to a distributed clock. This means, whenever the clock ticks, buffered inputs, if any, are distributed to the actor and it reacts. The tick function can be implemented using specific hardware for clocks, e.g., piezo-electric quartz crystals.

With this notion, there still come up problems, as event-driven reactions are delayed to the next tick.

1.6.3 Multiform Notion of Time

In multiform time, both events and a clock trigger reactions. It is implemented by combining the two approaches together.

There still remains the problem, that if the clock produces not enough ticks, internal reactions are delayed too much, which can be dangerous in the real-time systems context.

1.6.4 Eager Semantics

The best model should trigger reactions as soon as possible, in the timing relevant point in time, as defined in synchronous reactive semantics. This ideal is also called *eager semantics*.

Implementing eager semantics has a lot of requirements to make them realistically feasible for real-time systems. The main step in doing this implementation is using *dynamic ticks*.

Dynamic ticks are an extension of multiform time, where the clock additionally stores two values, ΔT the time since last tick and sleepT requested delay until next tick. The first is calculated on each tick, when reacting due to an event. The latter is set on each reaction based on information about the automata in its current state. If, while waiting, an event happens, and the sleep is interrupted, sleepT is changed based on ΔT .

Chapter 2

Design of Embedded Systems

In this chapter, we go over different practices and technologies regarding embedded systems. Most notably, sensors, actuators, embedded processors, memory architectures, input output (IO), scheduling, and a bit of Lingua Franca—an embedded systems programming language.

2.1 Sensors and Actuators

Sensors are devices which measure a physical quantity, while actuators modify a physical quantity. One vitally important sensor is the *accelerometer*, which measures acceleration, but it can also be used to measure the tilt of an embedded system. To achieve this, optical gyroscopes can also be used. Accelerometers can also be used to track a rough estimation of position based on measured accelerations, this is called an *inertial navigation system* (INS).

A problem all sensors have is, that they are notoriously known for *drift*, which makes calibration vitally important. To analyse this problem and other inaccuracies of a sensor, we define the *affine sensor model*.

Definition 2.1.1 (Affine Sensor Model). The affine sensor model defines physical properties as values $x \in \mathbb{R}$ and sensors as so-called *affine* linear functions $F : \mathbb{R} \rightarrow \mathbb{R}, x \mapsto ax + b$, with $a, b \in \mathbb{R}$, where a is called *sensitivity* and b is called *bias*.

To further enhance the realism of our model, we consider a *saturated* range $L, H \in \mathbb{R}, L \leq H$, and define the sensor function as

$$F : \mathbb{R} \rightarrow \mathbb{R}, x \mapsto \begin{cases} ax + b & \text{if } x \in [L, H] \\ aH + b & \text{if } x > H \\ aL + b & \text{if } x < L \end{cases}.$$

As a last enhancement, we consider the *precision* $p \in \mathbb{R}$ of the sensor, which is the smallest absolute difference between sensor readings, thus an even more realistic function is $F'(x) = p \cdot \lfloor F(x)/p \rfloor$, which clamps everything into a partitioned interval.

The dynamic range $D := \frac{H-L}{p}$ is often notated in decibels $D_{dB} := 20 \log_{10} D$. A quantized signal like F' often is a $n \in \mathbb{N}$ bit signal, which results in $D_{dB} := 20n \log_{10} 2 \approx 6n$.

When using sensors, it is advised to carefully use them if they are prone to aliasing. Alternatively, use higher-precision sensors.

Another way to handle sensor imprecision is to use multiple different sensors for the same quantity, which is called *redundancy*.

A classic way to drive actuators is pulse-width modulation (PWM), which uses repeated small pulses of the same voltage. Increasing pulse frequency, increases delivered power to the actuators.

2.2 Memory Architectures

Memory is used for different tasks in embedded systems, e.g., storage or communication with sensors and actuators. Embedded systems memory is often more restrictive than general purpose memory in size, power, and reliability. There are different types of memory.

Non-volatile Preserves contents when power is off

Flash Electrically erasable programmable read-only memory. Erases a block at a time. There is a limited number of write/erase cycles, i.e., around 100000.

Disks Too slow for embedded systems.

Volatile Loses contents when power is off

Registers Fastest access time possible, extremely expensive and power hungry. Used directly inside processors.

SRAM Static random-access memory. Very fast, deterministic access time. Expensive and power hungry. Used for (processor) caches and scratchpads.

DRAM Dynamic random-access memory. Slower than SRAM, but less power hungry and expensive.

Typically, embedded systems chips use memory-maps to partition all memory and sensor/actuator interfaces into different address spaces. E.g., using `0x0000` to `0x1FFF` for flash, `0x2000` to `0x4FFF` for DRAM, and `0x5000` to `0x5FFF` for SRAM.

Sometimes controllers use an e.g., 8-bit architecture, but 16-bit addresses. In this case, the controller needs two cycles to determine the position in memory for an address.

It is possible to manage the program memory in different methods. C programs typically use three different allocation methods for this.

Static Memory The compiler chooses the address at which to store a variable. A variable is static, if it is a global variable, or when it is declared **static** inside the scope of a function.

Stack The program dynamically allocates the variable using a LIFO strategy, pushing new allocations when entering a scope and popping on leaving a scope. Thus in C, stack-allocated variables are any non-static variables in a function scope.

Heap The developer manually dynamically allocates and deallocates memory on an extending structure. This is done by allocating address spaces with **void*** `malloc(size_t size)`, and deallocating it with **void** `free(void* ptr)`.

Sometimes stack and heap grow in the direction of the other, which limits (process) memory, but on embedded systems, a program typically can use all memory, as neither operating system nor processes exist.

Another topic to note is memory hierarchy. Most systems implement this using a linear hierarchy: Flash can be read into memory, memory caches parts of page lines, and registers load individual values. Scratchpads are developer-managed and typically used like cache.

2.3 Input and Output

Communication between different components and devices in an embedded system is often an IO task. Most microcontroller board realize this using general-purpose input/output (GPIO) pins, which can be used to read and write small voltage signals. Often, specific GPIO pins are reserved for specific purposes, such as *ADC*, *UART*, *I2C*, *SPI*, PWM, or serve to power electrical circuits using a higher voltage and ground.

There are two different general methods for wired interfaces: parallel connections, where each wire transfers one bit, and serial connections, where each wire serves one direction. There also exist hybrid approaches, but generally, serial transfers are preferred because of different problems with parallel transfers, i.e., interference or needed synchrony.

An example serial standard is RS-232, where positive and negative voltage encodes zeroes and ones. A transfer is done using a starting zero bit, data fragment (typically one byte) with LSB first, and a trailing one bit. Sending, e.g., 4 bytes, would be done by streaming each individual byte according to the format. The advantage of RS-232 is that it does not need a common clock between sender and receiver. It only needs the receiver clock to be faster than the frequency between bit signals without missing data.

Universal Serial Bus (USB) encodes data using two logical signals, D- and D+. Using this, there are four different combinations, which can be used to model different packet sizes, as there is an individual combination for the start and end of packet. Another difference is, that

the receiver signals the sender to send data and synchronisation is implemented¹.

Software IO input interfacing is implemented using two different approaches: *polling* and *interrupts*.

2.3.1 Polling

In polling, the active thread periodically looks in the IO interface, and uses and input if it is available. While it is simple to implement, the problem with polling is, that it either needs busy wait, or use repeated sleeps. Either, a lot of CPU time is wasted or data may be missed if not carefully implemented.

2.3.2 Interrupts

Interrupts prevent busy waiting by putting the active thread into a blocked IO mode, until hardware alerts the processor that input is ready. Upon the interrupt is triggered, an interrupt service routine is called, which needs to have been previously registered.

A big advantage of this is, that the program can do other things while waiting on the interrupt, as we are not restricted to one thread. Interrupts can also be used for timer interrupts, where an Interrupt service routine is triggered according to a periodic timer, e.g., blink every two seconds.

It has to be noted though to be careful with interrupts, as inter-thread interaction is non-deterministic and dangerous without semaphores or locks.

2.4 Concurrency

As seen in the last section, multitasking is unavoidable in many real-time systems. On that notion, it is important to differentiate between *concurrency*, the conceptual simultaneous execution, and *parallelism*, where simultaneous execution on distinct physical hardware is considered. A parallel execution, e.g., two threads on different cores, is concurrent, while a concurrent execution, e.g., two processes being scheduled on the same core, does not have to be parallel.

In POSIX/GNU threads are sequential procedures that share memory. The standard method to create threads is using the POSIX Threads (PTreads) API, where, most notably, a `void* _(void*)` function pointer, and some `void*` argument is passed to `pthread_create`. Waiting until a thread terminates is done using `pthread_join`.

A more modern framework for threads is *OpenMP*, which is generally considered to be more handy, as it implements a lot of useful tools, e.g., reductions or dependencies.

¹A great explanation on the implementation of the USB protocol is found here: https://en.wikipedia.org/wiki/USB_communications#Line_transition_state.

A key takeaway is, that threads are dangerous to use—they introduce non-deterministic properties in a wanted-to-be deterministic system. From the perspective of two threads using the same memory, when one changes something the other might use in some time its entire universe suddenly changes. Handling this nondeterminism is done with executive constrains, e.g., mutexes or limited access.

Another method to implement concurrency is by starting with a solid foundation, and only applying extensions, which are known to be well-behaved. This is the approach in many embedded systems programming languages, e.g., *Lingua Franca* or *SCCharts*.

Something to note is also, that the previously defined behaviour also needs to be implemented on the device, side of things. Before programming an embedded system, one needs to implement a microkernel, which handles operating systems side of things and implements every feature the programmer needs. An example of such a microkernel is the POSIX compliant Linux kernel based *Zephyr* micro operating system.

While implementing a microkernel, thread scheduling is an important and hard task.

2.4.1 Scheduling

In this section, we will go over different scheduling algorithms and strategies, to decide which scheduling is fit for embedded- and real-time systems.

Preemptive scheduling In preemptive scheduling all threads have priorities. In every moment the scheduler lets the non-blocked thread with highest priority execute.

Non-preemptive scheduling In non-preemptive scheduling algorithms, a task, e.g., a thread, is fragmented into jobs, which the scheduler picks according to some strategy.

There are a lot of different strategies for scheduling algorithms. We will define a scheduler to be *optimal*, iff it finds a schedule according to a *feasibility* criteria if one exists.

2.4.1.1 Rate Monotonic Scheduling

Rate Monotonic Scheduling (RMS) is a preemptive scheduling algorithm which assumes a simple process model with n independent tasks, period (deadlines) $T_1, \dots, T_n \in \mathbb{R}$, known worst-case execution times $C_1, \dots, C_n \in \mathbb{R}$, and negligible context switch time. RMS works by defining priorities by inverse period, and picking the highest priority task—the task with soonest deadline, and interrupting lower-priority tasks.

In RMS, feasibility is defined to mean, that every task completes before its period. It can be shown, that if any non-RMS schedule is feasible, the RMS schedule is also feasible. For this, the worst case is considered, where a low priority task starts when a high priority task is active. In that case a non-RMS schedule would pick the low-priority task over the high-priority task and result in total computation time $C_1 + C_2$, where $T_2 < T_1$. The schedule exists if $C_1 + C_2 \leq T_2$, and in that case, the RMS schedule is also feasible.

It can be shown, that if the average utilisation $\mu := \sum_{i \in [n]} C_i/T_i$ is smaller or equal to $n(2^{1/n} - 1)$, there always exists a feasible schedule—this is called the *utilization-based schedulability theorem* by Liu and Layland. If $\mu > m$, where $m \in \mathbb{N}$ is the number of processors, there can not be a feasible RMS schedule. Thus, as μ is bound by one in the case the inequality holds, an RMS schedule is always guaranteed.

If $\mu \leq 1$ and each task period is a multiple of the period of the next higher priority task, schedulability is also guaranteed.

2.4.1.2 Earliest Due Date

Earliest Due Date (EDD) is a non-preemptive scheduling algorithm, which executes tasks in order of non-decreasing deadlines. The difference to RMS is, that there are no task periods, but only deadlines, and that the algorithm is dynamic—when the active task finishes, the algorithm schedules the task with the smallest deadline.

This behaviour can be implemented using a minimum heap priority queue which stores tasks using the deadline as a key.

We define *maximum lateness* as $L_{\max} = \max_{i \in [n]} \{F_i - T_i\}$, where F_i is the calculated finishing time of task i . For non-preemptive scheduling, EDD minimizes maximum lateness.

2.4.1.3 Earliest Deadline First

Earliest Deadline First (EDF) is a preemptive version of EDD—on each scheduling event, i.e., a task finishes or some task changes blocked state, the scheduler schedules the task with minimal deadline. Another extension is, that tasks may arrive at any time, which means, that if an earlier due task arrives, the scheduler will change to that task.

The drawback of previously discussed algorithms is, that they are not optimal under precedence constraints—when a task can only be started, if it is dependent on another task.

EDF can be modified to meet precedence constraints by modifying deadlines of dependencies.

2.4.1.4 Latest Deadline First

Latest Deadline First (LDF) is a variation, where the schedule is built by always scheduling the leaf node from the dependency graph with the latest deadline last. LDF is optimal under precedence constraints.

2.4.2 Scheduling Anomalies

Using some form of mutex between threads complicates scheduling. When a low-priority task accesses a critical section, any high-priority task, which wants to access the section, stays blocked if the lower-priority task is blocked by a third task. In this case, the high-priority task

interchanges priority with the third task. This phenomenon is called *priority inversion*—the high-priority task is now dependent of the low-priority task, which is blocked by the third task.

Priority inversion can be mitigated by using the Priority Inheritance Protocol (PIP), which inherits the high priority to a lower-priority task, if it owns the lock on a critical section, which the high-priority task wants to acquire.

Another scheduling anomaly is a *deadlock*, in which a cyclic dependency between tasks causes an infinite suspension.

Deadlocks can be prevented using the Priority Ceiling Protocol (PCP), which assigns each lock a *priority ceiling* equal to the highest-priority task that can lock it. A task can only acquire a lock if its priority is strictly higher than the priority ceilings of all locks currently held by other tasks.

2.4.2.1 Richards Anomalies

A special sort of scheduling anomalies are *Richards anomalies*, named after Richard Graham. They occur when using a priority based scheduling algorithm on multicore processors. If tasks are scheduled as soon as possible based on their priority and dependency, increasing the number of available cores, reducing task computation time, or weakening precedence constraints might result in a worse schedule!

The takeaway of these anomalies are, that scheduling algorithms are brittle—small changes can have big and unforeseen consequences!

Chapter 3

Analysis and Verification

In this chapter we will use the methodology developed in Chapter 1 to evaluate model-based designs of embedded systems. For this, we first have to define what to evaluate for—what makes a model *correct*.

3.1 Correctness and Model Checking

Definition 3.1.1 (Correctness, Specification, Validation, and Verification). We define a model to be *correct* if it meets the right *specifications* in its operating environment.

The specification of a model is a mathematical statement of the design objectives—the desired properties of the system.

To *validate* a model is to prove if its specification meets the environments needs in the first place—you *build the right thing*.

Finally, the *verification* of a model is to check if the system has been build according to its specifications—you *build the thing right*.

As previously hinted, the goal of a model-based design approach is to not only synthesise software for embedded systems from a model, but also automatically verify it in doing so. The latter can be done with *model checking*, a process which determines whether a system satisfies a formal specification. The specification is a *temporal logic* formula, which expresses the desired systems properties over time. To define such formulas, we have to formally define *execution traces* and *propositional logic formulas*.

Definition 3.1.2 (Execution Traces and Propositional Logic). An execution trace of an SM is a sequence of the form $q_0, q_1, q_2, \dots$ with $q_i = (x_i, s_i, y_i)$ with x_i being the input valuation, s_i the state, and y_i the output valuation at step i .

A propositional logic formula is an extended version of the previously defined guards with s being another atomic statement, meaning that the SM is in state s . A more complicated

formula is e.g., $s_0 \wedge x = 1 \wedge \neg y$, meaning that s_0 is the active state, the input x is present with value 1 and the output is not present during the reaction.

We define a propositional logic formula to *hold* for a trace $q_0, \dots$ iff it holds for q_0 .

The reason in the last definition is, that we can define adequate propositional logic formulas to reason, that the SM must be correct starting from the first step of the trace.

There are two main variants of temporal logic formulas we will go into, linear temporal logic (LTL), and real-time temporal logic.

3.1.1 Linear Temporal Logic

LTL formulas are an extension of propositional logic formulas with the following grammar.

Definition 3.1.3 (Linear Temporal Logic Formula). An LTL formula extends propositions with four operators. The simplest LTL formula is just a proposition p .

For LTL formulas ϕ, ϕ_1, ϕ_2 , we define the following LTL formulas for a trace $q_0, q_1, \dots$

- $\mathbf{G}\phi$ ϕ holds for every suffix of $q_0, q_1, \dots$
- $\mathbf{F}\phi$ ϕ holds for some suffix of $q_0, q_1, \dots$
- $\mathbf{X}\phi$ ϕ holds for $q_1, \dots$
- $\phi_1 \mathbf{U} \phi_2$ ϕ_1 holds for all suffixes of $q_0, q_1, \dots$ until a suffix for which ϕ_2 holds.

Thus, $\mathbf{G}\phi$ is equivalent to mean, that ϕ holds for all steps of the trace, while $\mathbf{F}\phi$ only needs one step to hold to fulfil the formula. $\mathbf{GF}\phi$ means, that ϕ holds infinitely often. $\mathbf{FG}\phi$ means, that eventually ϕ holds for every step. $\mathbf{G}(\phi \Rightarrow \mathbf{F}\psi)$ means, that after ϕ holds, ψ must hold eventually. $\mathbf{G}(\phi \Rightarrow \mathbf{XX}\psi)$ means, that after ϕ holds, ψ holds exactly two reactions after.

Lemma 1 (Temporal Operators and Relationships). *For some trace and LTL formula ϕ , we have*

$$\mathbf{G}\phi = \neg \mathbf{F} \neg \phi, \quad (3.1)$$

$$\mathbf{F}\phi = \text{true} \mathbf{U} \phi, \quad (3.2)$$

$$X \text{ can not be expressed in terms of } \mathbf{G}, \mathbf{F}, \text{ or } \mathbf{U}. \quad (3.3)$$

Real-time temporal logic redefines the syntax of $\mathbf{F}$ to uphold deadlines, e.g., $\mathbf{F}_{\leq 2.5\text{ms}}$ means, that the following formula must hold within 2.5 ms of the trace.

3.2 Equivalence and Refinement

Another analysis goal for this chapter is the comparison of different models. There are different equality metrics, which can be used to compare two models.

Definition 3.2.1 (Type Equivalence). Two actors are *type equivalent* if they have the same input and output ports, which have the same sets of possible input and output valuations. In other words, they typecheck.

Definition 3.2.2 (Language Equivalence). Two actors are *language equivalent* if for every input sequence they can produce the same output sequence.

Unequivalent machines can still be *refinements* of each other, which is understood as a onesided equivalence. E.g., a machine can be a *type*, *language*, or *simulation refinement* of another machine, if it is type compatible, produces only outputs which the other can produce given the same input sequence, or produces the same output values on each reaction.

To formally define the refinements, recall SMs A and B with with states S_A, S_B , initial states I_A, I_B , input/output valuation sets V_A, V_B for set of input signals P_A, P_B , set of output signals Q_A, Q_B , and update functions u_A, u_B .

Definition 3.2.3 (Type Refinement). B is a type refinement of A if

$$P_B \subseteq P_A \quad B \text{ does not need all input signals,} \quad (3.4)$$

$$Q_A \subseteq Q_B \quad B \text{ may have more output signals,} \quad (3.5)$$

$$\forall p \in P_B : V_{Ap} \subseteq V_{Bp} \quad \text{input valuations typecheck,} \quad (3.6)$$

$$\forall q \in Q_B : V_{Bq} \subseteq V_{Aq} \quad \text{output valuations typecheck.} \quad (3.7)$$

In this case, B can replace A without causing a type conflict.

Definition 3.2.4 (Behaviour and Language Refinement). The *behaviour* of a SM is a tuple of input signals and produced output signals, such that the output signal matches the output sequence generated by an input sequence. The *language* set $L(\cdot)$ of a SM is the set of all behaviours.

B is a language refinement of A if $L(B) \subseteq L(A)$. I.e., B can replace A without producing anything A could not have produced.

Definition 3.2.5 (Simulation Relation / Simulation Refinement). A *simulates* B , if there is a relation $S \subseteq S_B \times S_A$ which

$$(I_B, I_A) \in S \quad (3.8)$$

$$\forall (s_B, s_A) \in S, i \in V_B^{in}, (s'_B, o) \in u_B(s_B, i) : \exists (s'_A, o) \in u_A(s_A, i) : (s'_B, s'_A) \in S \quad (3.9)$$

In other words, A simulates B if A 's initial state simulates B 's, and for each possible input B can have, the resulting state is also simulated by A and has the same output. S serves as a composed state machine of A and B that exists exactly when A simulates B .

Because A must only accept inputs which B accepts, we can show that $L(B) \subseteq L(A)$ in this case. Thus, if A simulates B , B is a simulation refinement of A and B is a language refinement of A .

Definition 3.2.6 (Bisimulation). An actor M_2 is *bisimilar* to actor M_1 if for every state of M_1 there exists a state in M_2 which reacts exactly the same way.

For deterministic SMs, language equivalence and bisimilarity are the same, but this is not the case in nondeterministic SMs.

The formal definition resembles the simulation relation, but with the difference, that the simulation must simulate both SMs at the same time. This is harder to that A and B simulate each other, as the simulation relations may not equal, which is forced in this definition.

3.3 Reachability Analysis and Model Checking

The goal in this section is to check for SMs if a target state s is reachable in the environment from initial state I —this is called a *reachability problem*.

The approach to do this is to create SMs for the system under development (SUD) and environment, and composing them, such that input and output signals are connected. This composed actor now has to be analysed. In this case, the system is called *closed* as the environment model provides all inputs and takes all outputs of the SUD SM.

Another analysis goal is to define an LTL formula for the composed actor, and automatically generate proofs/refutations.

Another task in the automated analysis is the search for simpler refinement actors to reduce state space and make other analyses easier.

3.3.1 Graph Traversal Algorithms

The problem with naive DFS and BFS is, that the runtimes and memory requirements are just too extreme for big state machines. The advantage of BFS over DFS is, that it finds the quickest path to a failure, if it exists.

There is a nice variant of BFS which can be used to represent everything cleaner and with less memory. If the graph to discover has nodes $[n]_0$, we can store a binary number $\mathfrak{m} \in \{1, 0\}^n$ which represents the current frontier in the search. Checking if a node i was already discovered can be done with $\mathfrak{m} \&\& 2^i$, and in this way, all neighbours of all new frontier nodes can be checked in one boolean operation.

3.3.2 Using Temporal Logic

As another approach, reachability analysis for state s can also be implemented using LTL formulas with Fs , meaning, that for some suffix, i.e., eventually, s is reached.

LTL can also be used to specify the order in which some states are reached, e.g., $F(s_1 \wedge F(s_2 \wedge F(s_3 \dots)))$.

3.3.3 Liveness Properties and Finding Acceptance Cycles

As important as checking if bad conditions are not met, we can also check if positive conditions will be met. E.g., $\Phi := \text{FG}\phi$, after some time a desired ϕ is always met.

Such a check can also be found by defining the *system automaton* of Φ , constructing the synchronous *Büchi automaton* composition of both SMs, and checking if there does not exist a reachable infinite cycle where the ϕ state is reached. In this case, we can refute Φ .

3.4 Quantitative Analysis

Aside from checking correctness of the system, we also need to check if it computes solutions in finite time, and ideally, within a fixed time bound. Both are generally impossible, as the halting problem is undecidable.

The goals here are how to determine the best case execution time (BCET) and worst case execution time (WCET). Under certain program complexity assumptions, we can approximate how to find WCETs. E.g., using program path/control flow analysis to find the longest paths in the program, or processor behaviour analysis.

As not every control flow graph is a DAG, we also need to consider bounds on possible infinite loops. Consider if a loop may run infinite or has a finite bound. If there are none, artificially add bounds for variables to enforce a breaking condition in finite time.

One important thing to note, is that WCETs are not composable, i.e., two tasks after each other may have a bigger total WCET due to the environment. Additionally, there can always be timing anomalies, e.g., random cache misses causing delays, or hardware related issues.

There are different tools to come up with WCET estimations, e.g., Kiel Esterel Processor (KEP), which can be used in SCCharts.

3.4.1 Measuring Runtime

There are different practices on how to accurately measure the runtime during execution. One method is using a cycle counter which increments a register value on each operation. For 2 GHz processors, this method results in a wrap around time of 2.1 seconds for 32 bit registers and 292 years for 64 bit registers. It is possible to use a 64 bit counter with 32 bit registers using the Intel *Read Time Stamp Counter* which works by using two registers to store the upper and lower 32 bits.

3.5 Achieving High Code Quality

When developing embedded real-time systems, maintaining a high code quality is essential to guarantee safety and reliability. Systems which are running 24/7 should especially avoid long term problems such as memory leaks or stack overflows.

Developing code with high quality is done using different methods.

3.5.1 Testing

One method in increasing reliability is writing different tests for software. All tests should be automatic, thorough, repeatable, independent, and professional (A TRIP). Repeatably using tests to propose wanted behaviour and independently implementing the feature such that all tests pass is called test-driven development (TDD).

A better approach is behaviour-driven development (BDD). The difference between the two is, that in BDD class behaviour should be expressed very naturally and descriptive in a test. In TDD the focus is mainly on specific (technical) methods.

3.5.1.1 Behaviour-Driven Development

BDD defines a workflow pipeline which goes from a top view into the concrete implementation steps.

1. Business goals: State the main goals from a business-side of view.
2. Features: Consult domain expert, developers, and testers to come up with features which contribute to the business goals.
3. Examples: Illustrate features with multiple examples.
4. Executable specifications: Derive high-level specifications, e.g., general/class behaviour, from examples.
5. Low-level specifications: Derive low-level specifications, e.g., unit tests, from high-level specifications. *Given, when, then*.
6. Application code: Start developing the code.
7. Testing: Automatically validate working features using specifications.

There are different sorts of development tests which can be used to implement BDD.

Unit tests Assumptions, test call, and validations can be split into different submethods in a test.

Mocking Define mocker behaviour in all test submethods and use BDD-flavored variants, e.g., *AssertJ* with *BDDAssertions*.

Automatic tests Generate many possible cases inside one test. If possible implement these tests syntactically as close to its high-level specifications as possible. *Cucumber* is a tool which helps in this step to automatically validate software based from its plain text specifications. This works by generating automatic tests from a domain specific language implemented by the tester, which accepts the plain text specification.